

Curso de Go

Aula 5 - Funções e múltiplos retornos

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 `código_` [3]
C/B [BRAZUCA]





Nessa Aula nos Vamos Ver

- **Definição de funções e parâmetros nomeados**
- **Retorno múltiplo (idiomático em Go)**
- **Closures e funções como valor**
- **Parte Prática**



O que são funções

- **Funções são blocos de código reutilizáveis**
- **Permitem:**
 - **Organizar o código**
 - **Evitar repetição**
 - **Melhorar a legibilidade**
- **Estrutura básica:**

Exemplo de código

```
func nome(parametros) retorno {  
    // código  
}
```



Função simples

- **Exemplo de função sem parâmetros**

```
package main  
import "fmt"
```

```
func saudacao() {  
    fmt.Println("Olá!")  
}
```

```
func main() {  
    saudacao()  
}
```

Sem uso de parênteses na condição



Função com parâmetros

- São funções com parâmetros que permitem entrada de dados

Exemplo de código

```
func somar(a int, b int) int {  
    return a + b  
}
```

```
func main() {  
    resultado := somar(2, 3)  
    fmt.Println(resultado)  
}
```

Exemplo de código simplificado

```
func somar(a, b int) int {  
    return a + b  
}
```




Retorno nomeado

- Go permite dar nomes aos valores de retorno da função.
- Esses nomes passam a funcionar como variáveis internas da função, podendo ser usados diretamente no corpo do código.

Exemplo de código

```
func dividir(a, b float64)
(resultado float64) {
    resultado = a / b
    return
}
```

- Como funciona:
 - O resultado já é declarado automaticamente como variável local
 - O return pode ser usado sem argumentos, pois ele retorna os valores nomeados
 - Isso pode deixar o código mais limpo em funções simples



Retorno nomeado

- **Boas práticas:**
 - **Use quando o nome do retorno ajuda a entender o que a função entrega**
 - **Evite em funções longas ou complexas, pois pode dificultar a leitura**
 - **Prefira retornos explícitos quando houver múltiplos valores ou lógica mais elaborada**
- **Resumo: melhora a clareza quando bem usado, mas pode prejudicar a legibilidade se usado em excesso**



Retorno múltiplo

- Uma das características mais importantes e idiomáticas de Go é permitir que funções retornem múltiplos valores. Isso é muito usado para retornar resultado + status ou resultado + erro.

Exemplo de código

```
func dividir(a, b float64) (float64, string) {  
    if b == 0 {  
        return 0, "erro: divisão por zero"  
    }  
    return a / b, "ok"  
}
```

```
func main() {  
    resultado, status := dividir(10, 2)  
    fmt.Println(resultado, status)  
}
```

- Como funciona:
 - A função retorna dois valores ao mesmo tempo
 - Na chamada, você precisa capturar ambos
 - A ordem dos retornos deve ser respeitada



Retorno múltiplo

- **Forma idiomática (mais usada em Go):**

```
func dividir(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("divisão por zero")  
    }  
    return a / b, nil  
}
```

- **Boas práticas:**
 - **Use retorno múltiplo para evitar estruturas complexas**
 - **Prefira error ao invés de string para erros**
 - **Sempre trate o erro ao chamar a função**
- **Resumo: retorno múltiplo é um dos pilares de Go para escrever código simples, claro e robusto**



Funções como valor

- Em Go, funções são tratadas como valores de primeira classe. Isso significa que você pode armazená-las em variáveis, passá-las como parâmetro e retorná-las de outras funções.

Exemplo de código

```
func somar(a, b int) int {  
    return a + b  
}
```

```
func main() {  
    f := somar  
    fmt.Println(f(2, 3))  
}
```

- Como funciona:
 - f passa a referenciar a função somar
 - Você pode chamar f como qualquer outra função
 - O tipo de f é: func(int, int) int



Funções como valor

- **Outro exemplo (passando função como parâmetro):**

Exemplo de código

```
func executar(op func(int, int) int) {  
    fmt.Println(op(5, 3))  
}
```

- **Boas práticas:**
 - Use funções como valor para criar código mais flexível e reutilizável
 - Muito útil para callbacks, estratégias e processamento genérico
 - Evite excesso de abstração em códigos simples
- **Resumo:** tratar funções como valores permite escrever código mais modular, reutilizável e expressivo em Go



Closures

- Closures são funções que capturam variáveis do ambiente onde foram criadas.
- Mesmo depois que a função externa termina, a função interna ainda mantém acesso a essas variáveis.

Exemplo de código

```
func contador() func() int {  
    i := 0  
    return func() int {  
        i++  
        return i  
    }  
}
```

- Ideia central:
 - A função interna “lembra” do valor de i
 - O estado é mantido entre chamadas



Como closures funcionam na prática

- **Uso da closure:**

```
func main() {  
    c := contador()  
  
    fmt.Println(c()) // 1  
    fmt.Println(c()) // 2  
    fmt.Println(c()) // 3  
}
```

- **O que acontece:**

- **contador()** cria uma nova variável **c**
- **Cada chamada de c()** altera o mesmo **i**
- **O estado não é recriado a cada chamada**

- **Importante:**

- **Cada nova chamada de contador()** cria uma nova closure independente



Quando usar closures

- Manter estado interno sem usar variáveis globais
- Criar funções personalizadas dinamicamente
- Implementar contadores, acumuladores, filtros

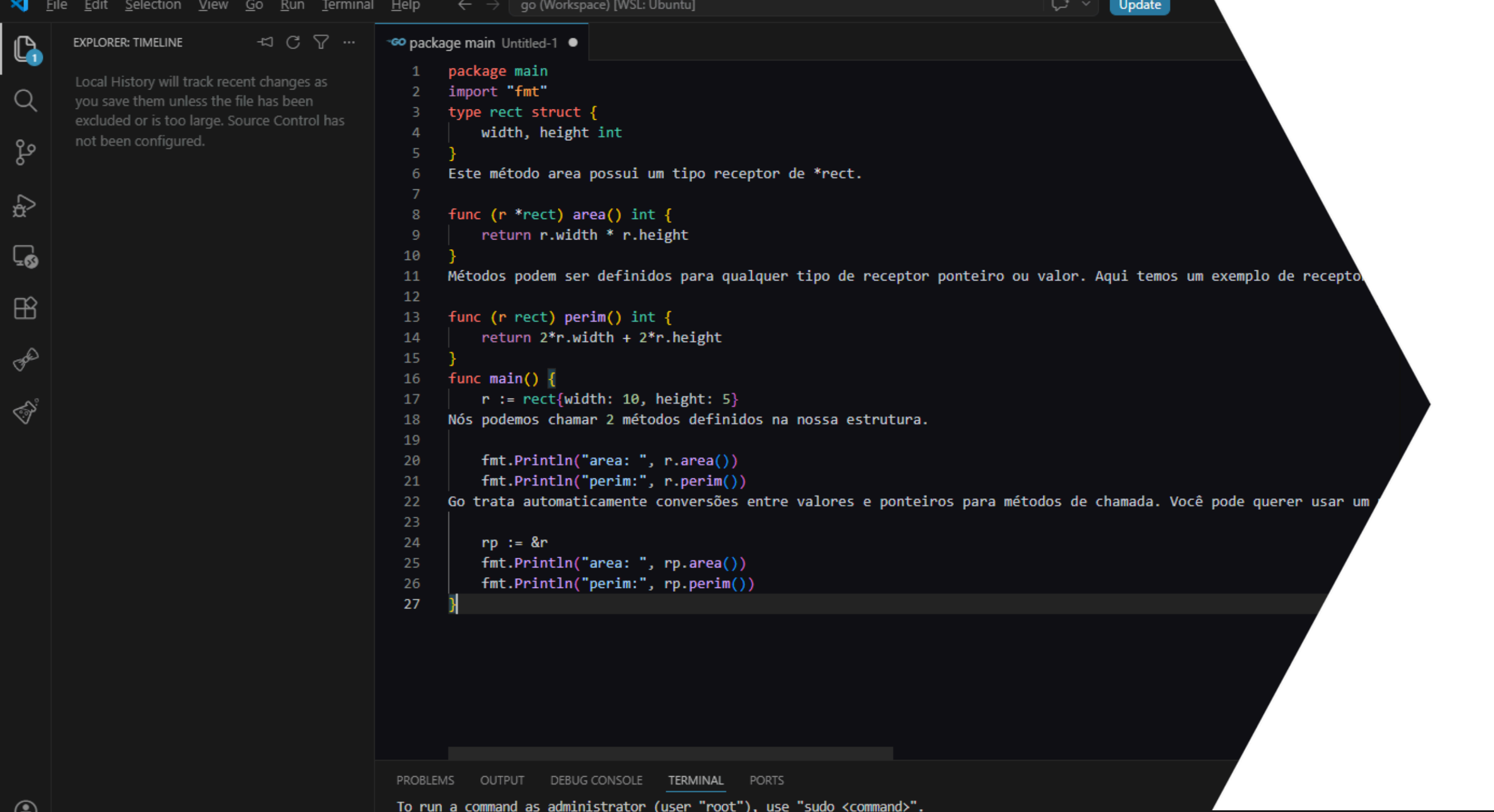
Exemplo de Código:

```
func multiplicador(fator int) func(int) int {  
    return func(x int) int {  
        return x * fator  
    }  
}
```

Uso:

```
dobro := multiplicador(2)  
fmt.Println(dobro(5)) // 10
```

- Na prática:
 - Closures ajudam a encapsular estado
 - Tornam o código mais flexível
 - Devem ser usadas com clareza para não confundir leitura



The screenshot shows a Go IDE with a dark theme. The Explorer sidebar on the left shows a file named 'package main' and a message: 'Local History will track recent changes as you save them unless the file has been excluded or is too large. Source Control has not been configured.' The main editor displays a Go program with the following code:

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

The bottom status bar shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active, displaying the message: 'To run a command as administrator (user "root"), use "sudo <command>".'

NÚCLEA
ACADEMY

Parte Prática

Praticar funções e closures